The marks mechanism*

Frank Mittelbach, LaTeX Project†

October 19, 2026

Abstract

Marks are used to communicate information about the content of a page to the output routine. For example, in order to construct running headers, the output routine needs information about which section names are present on a page, and this information is passed to it through the mark system. However, marks may also be used for other purposes. This module provides a generalized mechanism for marks of independent classes.

Contents

1 Introduction **2**

2 Design-level and code-level interfaces **2**
2.1 Use cases for conditionals 5
2.2 Understanding regions 5
2.3 Debugging mark code 7

3 Application examples **7**

4 Legacy LaTeX 2ε interface **7**
4.1 Legacy design-level and document-level interfaces 8
4.2 Legacy interface extensions 8

5 Notes on the mechanism **9**

6 Public interfaces for packages such as multicol **10**

7 Internal functions for the standard output routine of LaTeX **11**

8 The Implementation **12**
8.1 Allocating new mark classes 12
8.2 Updating mark structures 14
8.3 Placing and retrieving marks 22
8.4 Comparing mark values 24
8.5 Messages 25
8.6 Debugging the mark structures 25
8.7 Designer-level interfaces 27

*This file has version v1.1f dated 2026-06-26, © LaTeX Project.

†E-mail: latex-team@latex-project.org

9 LaTeX 2ε integration **28**
9.1 Core LaTeX 2ε integration 28
9.2 Other LaTeX 2ε output routines 31
9.3 Rollback information 32

1 Introduction

The TeX engines offer a low-level mark mechanism to communicate information about the content of the current page to the asynchronous operating output routine. It works by placing `\mark` commands into the source document. When the material for the current page is assembled in box 255, TeX scans for such marks and sets the commands `\topmark`, `\firstmark` and `\botmark`. The `\firstmark` receives the content of the first `\mark` seen in box 255 and `\botmark` the content of the last mark seen. The `\topmark` holds the content of the last mark seen on the previous page or more exactly the value of `\botmark` from the previous page. If there are no marks on the current page then all three are made equal to the `\botmark` from the previous page.

This mechanism works well for simple formats (such as plain TeX) whose output routines are only called to generate pages. It fails, however, in LaTeX (and other more complex formats), because here the output routine is sometimes called without producing a page, e.g., when encountering a float and placing it into one of the float regions. In that case the output routine is called, determines where to place the float, alters the goal for assembling text material (if the float was added to the top or bottom region) and then it resumes collecting textual material.

As a result the `\botmark` gets updated and so `\topmark` no longer reflects the situation at the top of the next page when that page is finally boxed.

Another problem for LaTeX was that it wanted to use several "independent" marks and in the early implementations of TeX there was only a single `\mark` command available. For that reason LaTeX implemented its own mark mechanism where the marks always contained two parts with their own interfaces: `\markboth` and `\markright` to set marks and `\leftmark` and `\rightmark` to retrieve them.

However, this extended mechanism (while supporting scenarios such as chapter/section marks) was far from general. The mark situation at the top of a page (i.e., `\topmark`) remained unusable and the two marks offered were not really independent of each other because `\markboth` (as the name indicates) was always setting both.

The new mechanism overcomes both issues:

- It provides arbitrarily many, fully independent named marks, that can be allocated and, from that point onwards, used.
- It offers access for each such marks to retrieve its top, first, and bottom values separately.
- Furthermore, the mechanism is augmented to give access to marks in different "regions" which may not be just full pages.

2 Design-level and code-level interfaces

The interfaces are mainly meant for package developers, but they are usable (with appropriate care) also in the document preamble, for example, when setting up special running

headers with fancyhdr, etc. They are therefore available both as CamelCase commands as well as commands for use in the L3 programming layer. Both are described together below.

`\NewMarkClass` `\NewMarkClass {⟨class⟩}`
`\mark_new_class:n` `\mark_new_class:n {⟨class⟩}`

Declares a new ⟨*class*⟩ of marks to be tracked by LaTeX. Each ⟨*class*⟩ must be declared before it is used.

Mark classes can only be declared before `\begin{document}`.

`\InsertMark` `\InsertMark {⟨class⟩} {⟨text⟩}`
`\mark_insert:nn` `\mark_insert:nn {⟨class⟩} {⟨text⟩}`

Adds a mark to the current galley for the ⟨*class*⟩, containing the ⟨*text*⟩.

It has no effect in places in which you can't place floats, e.g., a mark inside a box or inside a footnote never shows up anywhere.

If used in vertical mode it obeys LaTeX's internal `@nobreak` switch, i.e., it does not introduce a breakpoint if used after a heading. If used in horizontal mode it doesn't handle spacing (like, for example, `\index` or `\label` does, so it should be attached to material that is typeset.

`insertmark` `\AddToHook {insertmark} {⟨code⟩}`

When marks are inserted, the mark content may need some special treatment, e.g., by default `\label`, `\index`, and `\glossary` do not expand at this time (but only later if and when the mark content is actually used. In order to allow packages to augment or alter this setup there is a public hook `insertmark` that is executed at this point. It runs in a group so local modification to commands are only applied to the ⟨*text*⟩ argument of `\InsertMark` or `\mark_insert:nn`.

\TopMark ⋆ \TopMark [⟨region⟩] {⟨class⟩}
\FirstMark ⋆ \FirstMark [⟨region⟩] {⟨class⟩}
\LastMark ⋆ \LastMark [⟨region⟩] {⟨class⟩}
\mark_use_top:nn ⋆ \mark_use_top:nn {⟨region⟩} {⟨class⟩}
\mark_use_first:nn ⋆ \mark_use_first:nn {⟨region⟩} {⟨class⟩}
\mark_use_last:nn ⋆ \mark_use_last:nn {⟨region⟩} {⟨class⟩}

These functions expand to the appropriate mark ⟨*text*⟩ for the given ⟨*class*⟩ in the specified ⟨*region*⟩. The default ⟨*region*⟩ in the design-level commands is `page`. Note that with the L3 layer commands there are no optional arguments, i.e., both arguments have to be provided.

TEXhackers note: The result is returned within the `\unexpanded` primitive (`\exp_not:n`), which means that the ⟨*text*⟩ does not expand further when appearing in an `x`-type or `e`-type argument expansion.

The "first" and "last" marks are those seen first and last in the current region/page, respectively. The "top" mark is the last mark of the ⟨*class*⟩ seen in an earlier region, i.e., the ⟨*text*⟩ what would be "current" at the very top of the region.

Important!

The commands are only meaningful inside the output routine, in other places their result is (while not random) unpredictable due to the way LaTeX cuts text material into pages. There is, however, one exception: if you produce multiple columns using the multicol package, it is possible to retrieve mark values from the regions `first-column`, `last-column`, `mcol-1`, `mcol-2`,... directly after the environment has ended. This can, for example, be useful if a `multicols` has been be used inside a box.

Currently, ⟨*region*⟩ is one of `page`, `previous-page`, `column`, `previous-column`, `first-column`, `last-column`, and `mcol-1` (first column in a `multicols`), `mcol-2` (second column in a `multicols`), up to `mcol-20` (twentieth column in a `multicols`). See section 2.2 for discussion of how these regions behave and how one can make use of them.

\IfMarksEqualTF ⋆ \IfMarksEqualTF [⟨region⟩] {⟨class⟩} {⟨pos$_1$⟩} {⟨pos$_2$⟩} {⟨true⟩} {⟨false⟩}
\IfMarksEqualT ⋆ \mark_if_eq:nnnnTF {⟨region⟩} {⟨class⟩} {⟨pos$_1$⟩} {⟨pos$_2$⟩} {⟨true⟩} {⟨false⟩}
\IfMarksEqualF ⋆ \mark_if_eq:nnnnnnTF {⟨region$_1$⟩} {⟨class$_1$⟩} {⟨pos$_1$⟩}
\mark_if_eq:nnnnTF ⋆ {⟨region$_2$⟩} {⟨class$_2$⟩} {⟨pos$_2$⟩} {⟨true⟩} {⟨false⟩}
\mark_if_eq:nnnnnnTF ⋆

These conditionals allow you to compare the content of two marks and act based on the result. The commands work in an expansion context, if necessary.

It is quite common when programming with marks to need to interrogate conditions such as whether marks have appeared on a previous page, or if there are multiple marks present on the current page, and so on. The tests above allow for the construction of a variety of typical test scenarios, with three examples presented below.

The first two conditionals cover only the common scenarios. Both marks are picked up from the same ⟨*region*⟩ (by default `page`) and they have to be of the same ⟨*class*⟩.[1] The ⟨*pos$_i$*⟩ argument can be either `top`, `first`, or `last`.

Important to note is that the comparison is not with respect to the textual content of the marks but whether or not they originated from the same `\InsertMark` command (or the L3 layer version `\mark_insert:nn`).

If you wish to compare marks across different regions or across different classes, you have to do it using the generic test only available in the L3 programming layer or do it manually, i.e., get the marks and then compare the values yourself.[2]

[1] If an undeclared mark class is used the tests return *true* (not an error).

[2] If two undeclared mark classes are compared the result is always *true*; if a declared and an undeclared

2.1 Use cases for conditionals

However, the basic version is enough for the following typical use cases:

Test for at most one mark of class `myclass` on current page: If the first and last mark in a region are the same then either there was no mark at all, or there was at most one. To test this on the current page:

```
\NewMarkClass{myclass}
\IfMarksEqualTF{myclass}{first}{last}
               { <zero or one mark> }{ <two or more marks> }
```

Test for no mark of class `myclass` in the previous page: If the top mark is the same as the first mark, there is no mark in the region at all. If we wanted to do this test for the previous page:

```
\IfMarksEqualTF[previous-page]{myclass}{top}{first}
               { <no marks> }{ <at least one mark> }
```

Comparing `top` and `last` would give you the same result.

Test for zero, one, or more than one: Combining the two tests from above you can test for zero, one or more than one mark.

```
\IfMarksEqualTF{myclass}{top}{first}
               { <no marks> }
               {\IfMarksEqualTF{myclass}{first}{last}
                 { <exactly one mark> }{ <more than one mark> }}
```

If you need one of such tests more often (or if you want a separate command for it for readability), then consider defining:

```
\providecommand\IfNoMarkTF[2][page]{\IfMarksEqualTF[#1]{#2}{first}{last}}
```

2.2 Understanding regions

If a page has just been finished then the region `page` refers to the current page and `previous-page`, as the name indicates, refers to the page before the current page. This means you are able to access mark information for the current page as well as for the page before (as long as you are inside the output routine) without the need to explicitly save that information beforehand. The `page` region is the region that is most often queried, which is why commands like `\FirstMark` use that region by default.

In single column documents the `column` is the same as the `page` region, but in two-column documents (if not produced by `multicols`), `column` refers to the current column that just got finished and previous-column to the one previously finished. Code for running headers is (in standard LaTeX) evaluated only after both columns have been assembled, which is another way of saying that in that case `previous-column` refers to the left column and `column` to the right column. However, to make these somewhat easier to use, there are also aliased names for these two regions: `first-column` and `last-column`.[3]

mark class is used it is always *false*.

[3]The region is called "last-column" not "second-column" in anticipation of extending the mechanism to multiple columns, where first and last would still make sense. There aren't any `previous-first-column` and `previous-last-column` regions to access the corresponding columns from the previous page.

Note that you can only look backwards at already processed regions, e.g., in a `twoside` document finishing a recto (odd, right-hand) page you can access the data from the facing verso (left-hand) page, but if you are finishing a left-hand page you can't integrate data from the upcoming right-hand page. If such a scenario needs to be realized then it is necessary to save the left-hand page temporarily instead of finalizing it, process material for the right-hand page and once both are ready, attach running headers and footers and shipout out both in one go.[4]

The situation starts getting rather complex if you allow for multiple columns in the way they are supported by the multicol package. In this case you might have a variable number of "columns" on a single page to be shipped out. And even if not, then a `multicols` might start or end in the middle of the page; in either case, the regions `column` and `previous-column` become rather meaningless and you should therefore not use them.[5] Instead, the algorithm offers `mcol-1`, `mcol-2`, `mcol-3`, etc., to represent the columns in the `multicols` on the current page to be shipped out. If there is more than one `multicols` on the current page then in the output routine only the columns of the last one will be accessible.

These provisions cover, out of the box, a number of layouts and use cases, but obviously not all. However, more cases can be supported by storing away mark information during the processing. Here is the full algorithm:

- The `column` region is used by the "current column" that is being built (moving through all columns with `previous-column` trailing behind (to handle top marks properly).
- When the `multicols` starts, the `column` region is cleared, i.e., from that point on it looks as if there have not been any marks so far. This will make sure that the top mark in the first column is always empty.
- If the `multicols` extends beyond the current page, then the material designated for the current page is split into columns. The `column` region is used to represent each column in turn.
  - First we copy the current data from `column` to `previous-column`. Then the mark data from the current column is placed into the `column` region. Then we alias `column` to `mcol-1`.
  - These steps are repeated for all columns of the `multicols` environment.
  - Finally, the first and the last column of that page is also made available as `first-column` and `last-column`, respectively.
- All those marks inside any of the columns are also available in the `page` region. Thus, if you are interested in the top, first, or last mark of a specific class on the whole page you simply need to query for it in the `page` region.
- If the `multicols` continues across several pages then this algorithm above is repeated for each page, except that the `column` region is not cleared again. This means that the top mark of the first column of the next page will be the last mark of the last column from the previous page.

[4] As of now that scenario is not (yet) officially supported but it would be possible to achieve this using the shipout hooks to store the verso page and then on the next shipout use the hook to shipout both with running headers and footers attached.

[5] They return something, because they represent the last two columns of the `multicols` when you are inside the output routine, but that is obviously of little use.

- When the `multicols` finishes the remaining material for the current page is balanced to produce columns of roughly equal height.
- Again `column` and `previous-column` are used while this balancing happens and `mcol-1`, `mcol-2`, etc., are used to represent the column regions and `first-column` and `last-column` are set appropriately.
- Then the balanced set of columns is returned back to the page (since there may be space for further material). In addition, all marks inside that material are reinserted so that they become available in the `page` region.
- As a side effect, it is possible (and useful in certain circumstances) to query for mark classes directly after the `multicols` has ended without the need to be inside the output routine. The regions that can be queried this way are `mcol-1`, `mcol-2`, etc. (up to the number of columns the multicol had) and `first-column` and last-column.

2.3 Debugging mark code

`\DebugMarksOn`
`\DebugMarksOff`
`\mark_debug_on:`
`\mark_debug_off:`

`\DebugMarksOn ... \DebugMarksOff`

Commands to turn the debugging of mark code on or off. The debugging output is rather coarse and not really intended for normal use at this point in time.

3 Application examples

If you want to figure out if a break was taken at a specific point, e.g., whether a heading appears at the top of the page, you can do something like this:

```
\newcounter{breakcounter}
\NewMarkClass{break}
\newcommand\markedbreak[1]{\stepcounter{breakcounter}%
                           \InsertMark{break}{\arabic{breakcounter}%
                           \penalty #1\relax
                           \InsertMark{break}{-\arabic{breakcounter}}
```

To test if the break was taken you can test if `\TopMark{break}` is positive (taken) or negative (not taken) or zero (there was never any marked break so far). The absolute value can be used to keep track of which break it was (with some further coding).

to be extended with additional application examples

4 Legacy LaTeX 2ε interface

Here we describe the interfaces that LaTeX 2ε offered since the early nineties and some minor extensions.

4.1 Legacy design-level and document-level interfaces

`\markboth` `\markboth {⟨left⟩} {⟨right⟩}`
`\markright` `\markright {⟨right⟩}`

LaTeX 2ε uses two marks which aren't fully independent. A "left" mark generated by the first argument of `\markboth` and a "right" mark generated by the second argument of `\markboth` or by the only argument of `\markright`. The command `\markboth` and `\markright` are in turn called from heading commands such as `\chaptermark` or `\sectionmark` and their behavior is controlled by the document class.

For example, in the article class with `twoside` in force the `\sectionmark` will issue `\markboth` with an empty second argument and `\subsectionmark` will issue `\markright`. As a result the left mark will contain chapter titles and the right mark subsection titles.

Note, however, that in one-sided documents the standard behavior is that only `\markright` is used, i.e., there will only be right-marks but no left marks!

`\leftmark` ⋆ `\leftmark`
`\rightmark` ⋆ `\rightmark`

These functions return the appropriate mark value from the current page and work as before, that is `\leftmark` will get the last (!) left mark from the page and `\rightmark` the first (!) right mark.

In other words they work reasonably well if you want to show the section title that is current when you are about to turn the page and also show the first subsection title on the current page (or the last from the previous page if there wasn't one). Other combinations can't be shown using this interface.

The commands are fully expandable, because this is how they have been always defined in LaTeX. However, this is of course only true if the content of the mark they return is itself expandable and does not contain any fragile material. Given that this can't be guaranteed for arbitrary content, a programmer using them in this way should use `\protected@edef` and *not* `\edef` to avoid bad surprises as far as this is possible, or use the new interfaces (`\TopMark`, `\FirstMark`, and `\LastMark`) which return the ⟨*text*⟩ in `\exp_not:n` to prevent uncontrolled expansion.

4.2 Legacy interface extensions

The new implementation adds three mark classes: `2e-left`, `2e-right` and `2e-right-nonempty` and patches `\markboth` and `\markright` slightly so that they also update these new mark classes, so that the new classes work with existing document classes.

As a result you can use `\LastMark{2e-left}` and `\FirstMark{2e-right}` instead of `\leftmark` and `\rightmark`. But more importantly, you can use any of the other retrieval commands to get a different status value from those marks, e.g., `\LastMark{2e-right}` would return the last subsection on the page (instead of the first as returned by `\rightmark`).

The difference between `2e-right` and `2e-right-nonempty` is that the latter will only be updated if the material for the mark is not empty. Thus `\markboth{title}{}` as issued by, say, `\sectionmark`, sets a `2e-left` mark with `title` and a `2e-right` mark with the empty string but does not add a `2e-right-nonempty` mark.

Thus, if you have a section at the start of a page and you would ask for `\FirstMark{2e-right}` you would get an empty string even if there are subsections on that page. But `2e-right-nonempty` would then give you the first or last subsection

on that page. Of course, nothing is simple. If there are no subsections it would tell you the last subsection from an earlier page. We therefore need comparison tools, e.g., if top and first are identical you know that the value is bogus, i.e., a suitable implementation would be

```
\IfMarksEqualTF{2e-right-nonempty}{top}{first}
               { <appropriate action if there was no real mark> }
               {\FirstMark{2e-right-nonempty}}
```

5 Notes on the mechanism

In contrast to vanilla TEX, ε-TEX extends the mark system to allow multiple independent marks. However, it does not solve the `\topmark` problem which means that LATEX still needs to manage marks almost independently of TEX. The reason for this is that the more complex output routine used by LATEX to handle floats (and related structures) means that `\topmark(s)` remain unreliable. Each time the output routine is fired up, TEX moves `\botmark` to `\topmark`, and while ε-TEX extends this to multiple registers the fundamental concept remains the same. That means that the state of marks needs to be tracked by LATEX itself. An early implementation of this package used TEX's `\botmark` only to ensure the correct interaction with the output routine (this was before the ε-TEX mechanism was even available). However, other than in a prototype implementation for LATEX3, this package was never made public.

The new implementation now uses ε-TEX's marks as they have some advantages, because with them we can leave the mark text within the galley and only extract the marks during the output routine when we are finally shipping out a page or storing away a column for use in the next page. That means we do not have to maintain a global data structure that we have to keep in sync with informational marks in the galley but can rely on everything being in one place and thus manipulations (e.g. reordering of material) will take the marks with them without a need for updating a fragile linkage.

To allow for completely independent marks we use the following procedure:

- For every type of marks we allocate a mark class so that in the output routine TEX can calculate for each class the current top, first, and bottom mark independently. For this we use `\newmarks`, i.e., one marks register per class.
- As already mentioned firing up an output routine without shipping out a page means that TEX's top marks get wrong so it is impossible to rely on TEX's approach directly. What we do instead is to keep track of the real marks (for the last page or more generally last region) in some global variables.
- These variables are updated in the output routine at defined places, i.e., when we do real output processing but not if we use special output routines to do internal housekeeping.
- The trick we use to get correctly updated variables is the following: the material that contains new marks (for example the page to be shipped out) is stored in a box. We then use TEX primitive box splitting functions by splitting off the largest amount possible (which should be the whole box if nothing goes really wrong). While that seems a rather pointless thing to do, it has one important side effect: TEX sets up first and bottom marks for each mark class from the material it has split off. This way we get the first and last marks (if there have been any) from the material in the box.

- The top marks are simply the last marks from the previous page or region. And if there hasn't been a first or bottom mark in the box then the new top mark also becomes new first and last mark for that class.
- That mark data is then stored in global token lists for use during the output routine and legacy commands such as `\leftmark` or new commands such as `\TopMark` simply access the data stored in these token lists.

That's about it in a nutshell. Of course, there are some details to be taken care of—those are discussed in the implementation sections.

6 Public interfaces for packages such as multicol

The functions in this section are public so that packages can make use of them. However, this must be done with great care, e.g., `\mark_update_structure_from_material:nn` updates the global mark structure and can therefore be used only in places where such an update is meaningful, e.g., in special output routines. Elsewhere, a change to the mark structure would break the whole mechanism and querying the marks would return incorrect data.

`\mark_update_structure_from_material:nn` `\mark_update_structure_from_material:nn {⟨region⟩} {⟨material with marks⟩}`

Helper function that inspects the marks inside the second argument and assigns new mark values based on that to the ⟨*region*⟩ given in the first argument. For this it first copies the mark structure from ⟨*region*⟩ to `previous-`⟨*region*⟩ and then takes all last mark values currently in the region and makes them the new top mark values. Finally it assigns new first and last values for all mark classes based on what was found in the second argument.

As a consequence, the allowed values for ⟨*region*⟩ are `page` and `column` because only they have `previous-...` counterparts.

Another important aspect to keep in mind is that marks are recognized only if they appear on the top level, e.g., if we want to process material stored in boxes we need to put it unboxed (using `\unvcopy` etc.) into the second argument.

`\mark_copy_structure:nn` `\mark_copy_structure:nn {⟨alias⟩} {⟨source⟩}`

Helper function that copies all mark values in the ⟨*source*⟩ region to ⟨*alias*⟩, i.e., make the structures identical. Used to update the `previous-...` structures inside `\mark_update_structure_from_material:nn` and `first-column` and `last-column` structures inside the internal commands `\__mark_update_singlecol_structures:` or `\__mark__update_dblcol_structures:`.

`\mark_set_structure_to_err:n` `\mark_set_structure_to_err:n {⟨region⟩}`

Helper function that sets all mark values in the ⟨*region*⟩ to an error message. This is currently used for `last-column` at times where using marks from it would be questionable/wrong, i.e., when we have just processed the first column in a two-column document.

\mark_clear_structure:n `\mark_clear_structure:n {⟨region⟩}`

Helper function that sets all mark values in the ⟨*region*⟩ to empty. This is currently used for `column` when a multicol environment starts; this is because it wouldn't make sense if the top mark in the first column returned the last mark from a previous multicol (which may have been much earlier, with intermediate material).

\mark_get_marks_for_reinsertion:nNN `\mark_get_marks_for_reinsertion:nNN {⟨source⟩}`
`⟨token-list-var for collecting first marks⟩`
`⟨token-list-var for collecting last marks⟩`

Helper function for extracting marks that would otherwise get lost, for example when they are hidden inside a box. This helper does not update mark structures and can therefore be used outside the output routine as well.

It collects all the top-level marks from inside the ⟨*source*⟩ and then adds suitable `\mark_insert:nn` commands to each of the two token lists. These token lists can then be executed at the right place to reinsert the marks, e.g., directly after the box. This is, for example, going to be used[6] by multicol when a short balanced `multicols` is returned to the galley for typesetting.

If the ⟨*source*⟩ consists of a single vertical box (plus possibly followed by some glue but nothing else) then the box is unpacked and the top-level marks are collected from its content. However, if it is not a vertical box or there are other data then nothing is unpacked and you have to do the unpacking yourself to get at the marks inside.

It is quite likely that one only needs a single token list for returning the `\mark_insert:nn` statements. If that is the case this command may change to take only two arguments.

7 Internal functions for the standard output routine of LaTeX

The functions in this section are tied to the output routine and used in the interface to LaTeX 2ε and perhaps at some later time within a new output routine for LaTeX. They are not (yet) meant for general use and are therefore made internal, even though we already use them in multicol. Internal means that `@@` automatically gets replaced in the code (and in the documentation) so we have to give it a suitable value.

```
1 ⟨@@=mark⟩
```

__mark_update_singlecol_structures: `\__mark_update_singlecol_structures:`

LaTeX 2ε integration function in case we are doing single column layouts. It assumes that the page content is already stored in `\@outputbox` and processes the marks inside that box. It is called as part of `\@opcol`.

__mark_update_dblcol_structures: `\__mark_update_singlecol_structures:`

LaTeX 2ε integration function mark used when we are doing double column documents. It assumes that the page content is already stored in `\@outputbox` and processes the marks inside that box. It then does different post-processing depending on the start of the switch `\if@firstcolumn`. If we are in the second column it also has to update page marks, otherwise it only updates column marks. It too is called as part of `\@opcol`.

[6] Probably not before 2025, though.

8 The Implementation

```
⟨*2ekernel | latexrelease⟩
\ExplSyntaxOn
⟨latexrelease⟩\NewModuleRelease{2022/06/01}{ltmarks}
⟨latexrelease⟩                        {Marks~handling}
```

8.1 Allocating new mark classes

`\g__mark_classes_seq` A list holding all the mark classes that have been declared.

```
\seq_new:N \g__mark_classes_seq
```

`\mark_new_class:n` `\__mark_new_class:nn` A mark class is created by initializing a number of data structures. First, we get a register number to refer to the mark class. The new mark class is then added to the `\g__mark_classes_seq` sequence to be able to easily loop over all classes. Finally a number of top-level global token lists are declared that hold various versions of the mark for access.

```
\cs_new_protected:Npn \mark_new_class:n #1
{
  \seq_if_in:NnTF \g__mark_classes_seq {#1}
      {
        \msg_error:nnn { mark } { class-already-defined }
          {#1}
      }
      { \__mark_new_class:nn {#1} }
}
```

This is only available in the preamble.

```
\@onlypreamble \mark_new_class:n
```

The internal command carries out the necessary allocations.

```
\cs_new_protected:Npn \__mark_new_class:nn #1
{
⟨*trace⟩
  \__mark_debug:n { \iow_term:e { Marks:~new~mark:~#1~\msg_line_context: } }
⟨/trace⟩
```

Use the LaTeX 2_ε interface for now as the L3 programming layer doesn't have one for marks yet.

```
  \exp_args:Nc \newmarks {c__mark_class_ #1 _mark}
```

Remember the new class in the sequence.

```
  \seq_gput_right:Nn \g__mark_classes_seq {#1}
  \__mark_init_region:nn {page}{#1}
```

For the `page` region we also keep track of the `previous-page`.

```
  \__mark_init_region:nn {previous-page}{#1}
```

Same game for `column` and `previous-column`

```
  \__mark_init_region:nn {column}{#1}
  \__mark_init_region:nn {previous-column}{#1}
```

But for columns we also allocate token lists for the alias regions `first-column` and `last-column`.

```
  \__mark_init_region:nn {first-column}{#1}
  \__mark_init_region:nn {last-column}{#1}
```

To support multiple columns produced by the multicol package, we preallocate twenty alias regions (since this is the number of columns that multicol supports as a maximum). They are filled by copying the current `column` into the appropriate `mcol-...`.

```
%fmi  \__mark_init_region:nn {mcol}{#1}
%fmi  \__mark_init_region:nn {previous-mcol}{#1}
  \__mark_init_region:nn {mcol-1}{#1}
  \__mark_init_region:nn {mcol-2}{#1}
  \__mark_init_region:nn {mcol-3}{#1}
  \__mark_init_region:nn {mcol-4}{#1}
  \__mark_init_region:nn {mcol-5}{#1}
  \__mark_init_region:nn {mcol-6}{#1}
  \__mark_init_region:nn {mcol-7}{#1}
  \__mark_init_region:nn {mcol-8}{#1}
  \__mark_init_region:nn {mcol-9}{#1}
  \__mark_init_region:nn {mcol-10}{#1}
  \__mark_init_region:nn {mcol-11}{#1}
  \__mark_init_region:nn {mcol-12}{#1}
  \__mark_init_region:nn {mcol-13}{#1}
  \__mark_init_region:nn {mcol-14}{#1}
  \__mark_init_region:nn {mcol-15}{#1}
  \__mark_init_region:nn {mcol-16}{#1}
  \__mark_init_region:nn {mcol-17}{#1}
  \__mark_init_region:nn {mcol-18}{#1}
  \__mark_init_region:nn {mcol-19}{#1}
  \__mark_init_region:nn {mcol-20}{#1}
```

We also have to initialize the `saved-column` region that is used in multicol. Perhaps we should have a `\NewMarkRegion` so that it would be possible for other packages to add further regions. But let's wait and see if there is a real use case for new regions before making the interface more general.

```
  \__mark_init_region:nn {saved-column}{#1}
}
```

(*End of definition for* `\mark_new_class:n` *and* `\__mark_new_class:nn`. *This function is documented on page 3.*)

`\__mark_init_region:nn`
`\c__mark_empty_tl`

For each class (`#2`) and region (`#1`), we need three token lists: one for top, first, and last. The default value to be returned is "empty".

```
\cs_new_protected:Npn \__mark_init_region:nn #1 #2 {
  \tl_new:c { g__mark_#1_top_   #2 _tl }
  \tl_new:c { g__mark_#1_first_ #2 _tl }
  \tl_new:c { g__mark_#1_last_  #2 _tl }
  \tl_gset_eq:cN  { g__mark_#1_top_   #2 _tl } \c__mark_empty_tl
  \tl_gset_eq:cN  { g__mark_#1_first_ #2 _tl } \c__mark_empty_tl
  \tl_gset_eq:cN  { g__mark_#1_last_  #2 _tl } \c__mark_empty_tl
}
```

All marks will have an identification in the form of a number[7] that is incremented each time a mark insertion happens; therefore the initial empty values should also have such a number, so that data extraction will be uniform.

```
\tl_const:Nn \c__mark_empty_tl { \__mark_value:nn{0}{} }
```

(*End of definition for* `\__mark_init_region:nn` *and* `\c__mark_empty_tl`.)

8.2 Updating mark structures

`\l__mark_box`
`\l__mark_ii_box`
`\g__mark_tmp_tl`
`\g__mark_new_top_tl`

For some operations we need two temporary private boxes and two private global token lists.

```
\box_new:N \l__mark_box
\box_new:N \l__mark_ii_box
\tl_new:N  \g__mark_tmp_tl
\tl_new:N  \g__mark_new_top_tl
```

(*End of definition for* `\l__mark_box` *and others.*)

`\__mark_extract_and_handle_marks:nn`

This is the main macro to extract and handle marks inside some vertical material. It is used by `\mark_update_structure_from_material:nn` (for updating the mark structure for a region based on the marks found) and by `\mark_get_marks_for_reinsertion:nNN` (for extracting marks from some material and prepare for reinserting them later (e.g., out of a box that is placed as a box into the main galley).

```
\cs_new_protected:Npn \__mark_extract_and_handle_marks:nn #1#2 {
```

This macro expects code to handle extracted marks in its first argument and vertical material (not boxed or just consisting of a single vertical box) as its second. It extracts top-level mark information from `#2`, stores them as split marks and then calls `#1` to make use of this information.

If it finds a forced break in the material it removes it and then restarts the attempt without it.

We start with a group to keep most changes local.

```
  \group_begin:
```

Getting the first and last marks out of the material in `#2` is done by putting the material in a box and then doing a split operation to the maximum size possible (which hopefully gets us all of the content).[8] Because this action is used only to get the mark values, we don't want any underfull box warnings so we (locally) turn those off.

```
    \dim_set_eq:NN \tex_splitmaxdepth:D \c_max_dim
    \int_set_eq:NN \tex_vbadness:D      \c_max_int
    \dim_set_eq:NN \tex_vfuzz:D         \c_max_dim
```

There is a further complication: if the material contains infinite shrinking glue then a `\vsplit` operation will balk with a low-level error. Now pages or columns, which are our main concern here, can't have such infinite shrinkage if they are cut straight from the galley, however the use of `\enlargethispage` actually does add some at the very bottom (and also wraps the whole page into a box by itself, so if we leave it this way then a) we get this error and b) we don't see any marks because they are hidden one level down).

[7] There are a few cases where special identification strings are used, e.g., `2.09-compat`.

[8] With normal column material cut from the main galley we should always get all material in one go, but in certain situations, for example, in a multicols environment that contains some `\columnbreaks` a single split operation will not be enough. Thus, this is something we need to handle.

Another possible issue are packages or user code that place stray \vboxes directly into the main galley (an example is marginnote that attaches its marginals in this way). If such boxes end up as the last item on the page we should not unpack them.

All these issues need to be handled, which is done in __mark_prepare_and_extract:nn.

```
    \__mark_prepare_and_extract:nn {#1} {#2}
```

Once all mark classes have been processed, the data structures are updated and we can close the group, which undoes our local changes and retains only the global ones.

```
  \group_end:
}
```

(*End of definition for* __mark_extract_and_handle_marks:nn.)

__mark_prepare_and_extract:nn

This macro does the dirty work. It is not directly integrated in __mark_extract_and_handle_marks:nn because we may have to call it recursively if we find forced breaks.

```
\cs_new_protected:Npn \__mark_prepare_and_extract:nn #1#2 {
```

To handle the \enlargethispage case we do an \unskip to get rid of any glue that is present at the very end of the material and also check if we have then a \vbox as the last item and if so unpack that too, but only under certain conditions, see below. All this is temporary done in a group, just for getting the marks out, so it doesn't affect the final page production.

```
  \vbox_set:Nn \l__mark_box
    {
      #2
      \tex_unskip:D
      \box_set_to_last:N \l__mark_box
```

After having removed the last box from the current list (if there was one) we check whether the vertical list is now empty. If not, then the last box is definitely not the one from \enlargethispage and so we can, and should, leave it alone. Otherwise we check if this last box is a \vbox.

```
      \int_compare:nNnT \tex_lastnodetype:D < 0
        {
          \box_if_vertical:NT \l__mark_box
```

If it is, we unpack the box.

```
            { \vbox_unpack:N \l__mark_box }
        }
```

If it wasn't a vbox, it was either an hbox or there was no box. Given that we are only interested in the marks we don't need put it back in that case.

```
    }
```

We are now ready to \vsplit the box to get at the marks. If the box contains some infinite negative glue the TEX will produce an error complaining about it but it will correctly find the split marks. Given that we can't prevent that error, we hide it from the user and ensure that TEX doesn't stop. The error message still shows in the log, but even that is mitigated as best as possible—see the definition of __mark_vbox_set_split_to_maxdimen:NN for the tricks employed.

```
    \__mark_vbox_set_split_to_maxdimen:NN \l__mark_ii_box \l__mark_box
```

After splitting we check if there is anything left in `\l__mark_box`. If not then the above split has set some split marks that we can then use to finish the extraction:

```
    \box_if_empty:NTF \l__mark_box
      { #1 }
```

If we have a remainder after the split then this means that there was some forced break in the material. We get rid of that by combining the content of the two boxes and restart.

```
      {
⟨*trace⟩
        \__mark_debug:n { \iow_term:e
          { Marks:~ mark~ extraction~needs~ recursion~
            \msg_line_context: } }
⟨/trace⟩
        \__mark_prepare_and_extract:nn {#1}
          { \vbox_unpack:N \l__mark_ii_box
            \vbox_unpack:N \l__mark_box    }
      }
}
```

(*End of definition for* `\__mark_prepare_and_extract:nn`.)

`\__mark_vbox_set_split_to_maxdimen:NN` Split a box to get at its marks without pausing even if TEX is producing an error message because of infinite negative glue in the box. If there is such an error we ensure that it only shows up in the log but not on the terminal.

With a recent TEX engine that knows the primitive `\ignoreprimitiveerror` we can turn this error into a warning (simply by setting this primitive to the value 1).

```
\if_cs_exist:N \tex_ignoreprimitiveerror:D

\cs_new_protected:Npn \__mark_vbox_set_split_to_maxdimen:NN #1#2 {
  \tl_set:Ne \l__mark_saved_parameters_tl
    { \tex_ignoreprimitiveerror:D
      \int_use:N \tex_ignoreprimitiveerror:D
      \scan_stop:
    }
  \tex_ignoreprimitiveerror:D 1 \scan_stop:
  \vbox_set_split_to_ht:NNn #1 #2 { \c_max_dim }
  \l__mark_saved_parameters_tl
}
```

With older TEX engines we have to make use of David's hack below to render the error (fairly) harmless and prevent TEX from stopping. But, of course, the above solution is better because jumping over the error with a local change to the interaction mode still means that TEX thinks there was an error in the run so the return code is no longer 0 (and that might affect workflows that want to test for this).

```
\else:
```

The nice low-level hack by DPC records in the `.log` that a glue shrinkage error is harmless.

We disguise `\c_max_dim` in an odd looking csname, which then shows up as part of the display of an error message if that error happens. This csname forms part of the error display so what you get is something like

```
! Infinite glue shrinkage found in box being split.
<argument> Infinite shrink error above ignored !
l. ...  }
```

which hopefully makes it clear that the error is harmless and and should be ignored by the reader of the `.log`.

```
\cs_set_eq:cN {Infinite~shrink~error~above~ignored~!}\c_max_dim
```

The whole definition of `\__mark_vbox_set_split_to_maxdimen:NN` below is fully expanded, so we have to use a lot of `\exp_not:N` commands to prevent expansion where necessary.

```
\cs_new_protected:Npe \__mark_vbox_set_split_to_maxdimen:NN #1#2 {
```

We start by saving the current interaction and escape char settings.

```
  \tl_set:Ne \exp_not:N \l__mark_saved_parameters_tl
    {
      \tex_interactionmode:D
        \exp_not:N \int_use:N \tex_interactionmode:D \scan_stop:
      \tex_escapechar:D
        \exp_not:N \int_use:N \tex_escapechar:D \scan_stop:
    }
```

Then we change them so that no escape char is printed in the error message (accounts for the missing backslash in front of `Infinite shrink ...`) and we set the interaction to `\nonstopmode` so that the error (if any) just goes into the `.log` file and TEX doesn't stop at that point.

```
  \tex_escapechar:D     -1 \scan_stop:
  \tex_interactionmode:D 0 \scan_stop:
```

Then we do the splitting of the box to `\c_max_dim` to get at the marks. This may generate the error we are worried about, i.e., if the box contains infinite negative glue. However, TEX makes this glue finite and continues, which means we get our split marks which is really all we care about.

```
  \tex_setbox:D #1 \tex_vsplit:D #2 to
```

The `\use:n` may seem pointless, and it is to some extent, but we need it to get our disguised `\c_max_dim` displayed properly as part of the error message if there is one. Without it, the display would show only part of what we want it to show (try it).

```
      \exp_not:N \use:n {
        \use:c{Infinite~shrink~error~above~ignored~!}
      }
```

Finally, we change the escape char and the interaction mode back to what it was before:

```
  \exp_not:N \l__mark_saved_parameters_tl
}
\fi:
```

(*End of definition for* `\__mark_vbox_set_split_to_maxdimen:NN`.)

`\l__mark_saved_parameters_tl` The temporary variable used for resetting escape char and interaction mode.

```
\tl_new:N \l__mark_saved_parameters_tl
```

(*End of definition for* `\l__mark_saved_parameters_tl`.)

`\mark_update_structure_from_material:nn` This function updates the mark structures of a region. The first argument is the region to update and second argument receives the material that holds the marks. Out of this material we extract the first and last marks for all classes (if there are any) to do the assignments.

```
\cs_new_protected:Npn \mark_update_structure_from_material:nn #1#2 {
  \__mark_extract_and_handle_marks:nn
```

Once the marks can be extracted we update the structure from the split marks (code in `\__mark_update_structure_from_splitmarks:n`).

```
      { \__mark_update_structure_from_splitmarks:n {#1} }
      { #2 }
}
```

(*End of definition for* `\mark_update_structure_from_material:nn`. *This function is documented on page* 10.)

`\__mark_update_structure_from_splitmarks:n`

This macro is called after we have done a `\tex_vsplit:D` operation and the mark data is in the split marks.

```
\cs_new_protected:Npn \__mark_update_structure_from_splitmarks:n #1 {
```

The first thing we do is to copy the current region structure to `previous-...`; this leaves the current structure untouched so we can update it class by class (as is necessary).

```
  \mark_copy_structure:nn { previous-#1 } {#1}
```

After this action we can get first and last marks of the various classes through `\tex_splitfirstmarks:D` and `\tex_splitbotmarks:D`. So now we loop over all classes stored in `\g__mark_classes_seq`.

```
  \seq_map_inline:Nn \g__mark_classes_seq
            {
```

First action: get the last mark from the previous region, i.e., `previous-#1`. But because it is also still inside `#1`, at the moment we use that to construct the name because this is a tiny bit faster. Given that we need this value in various assignments we store it away which avoids unnecessary further csname generations.

```
              \tl_gset_eq:Nc \g__mark_new_top_tl { g__mark_#1_last_##1_tl }
```

This will first of all become the new top mark for the current class.

```
              \tl_gset_eq:cN { g__mark_#1_top_##1_tl } \g__mark_new_top_tl
```

Next action is to get ourselves the new last mark from the material supplied.

```
              \tl_gset:No \g__mark_tmp_tl
                { \tex_splitbotmarks:D \use:c { c__mark_class_##1_mark } }
```

If this mark doesn't exist then obviously neither does the first mark, so both become the last mark from the previous region. We have to be a little careful here: something like `\mark_insert:nn{foo}{}` adds an "empty" mark that should not be confused with no mark at all. But no mark in our material will result in `\g__mark_tmp_tl` being fully empty. This is why we have to make sure that "empty" from `\mark_insert:nn` only appears to be empty when typeset but fails the next test (see below how this is done).

```
              \tl_if_empty:NTF \g__mark_tmp_tl
                {
                  \tl_gset_eq:cN { g__mark_#1_last_ ##1_tl }
                    \g__mark_new_top_tl
                  \tl_gset_eq:cN { g__mark_#1_first_##1_tl }
                    \g__mark_new_top_tl
                }
```

If it wasn't empty, i.e., if it had a real value then we use this value for our new last mark instead.

```
                {
                  \tl_gset_eq:cN { g__mark_#1_last_##1_tl } \g__mark_tmp_tl
```

Because we had a last mark we also have a first mark (which might be the same, but might be not), so we pick that up and assign it to the appropriate token list. This explains why we first checked for the last mark because that makes the processing faster in case there is none.

```
                \tl_gset:co { g__mark_#1_first_##1_tl }
                  {
                    \tex_splitfirstmarks:D
                      \use:c { c__mark_class_##1_mark }
                  }
              }
          }
}
```

(*End of definition for* `\__mark_update_structure_from_splitmarks:n`.)

`\mark_get_marks_for_reinsertion:nNN` This function extracts the marks from the material in the first argument but it does not update any the mark structures. Instead, it collects the marks in the token lists given as the second and third argument, in such a way that they can be reinserted by just executing the token lists.[9]

```
\cs_new_protected:Npn \mark_get_marks_for_reinsertion:nNN #1#2#3 {
```

First we clear the temporary token lists as we haven't seen any marks yet.

```
  \tl_gclear:N \g__mark_first_marks_tl
  \tl_gclear:N \g__mark_last_marks_tl
```

Then we extract all top-level marks, thereby filling the token lists with suitable `\mark_insert:nn` calls.

```
  \__mark_extract_and_handle_marks:nn
```

The first argument holds the code used for filling the token lists and the second holds the material from which all marks should be extracted.

```
    \__mark_get_from_splitmarks:
    { #1 }
```

Finally, we copy the updated (or not updated) temporary token lists to the two that have been supplied when the function was called. By convention "get" operations return their values in local variables and `\__mark_extract_and_handle_marks:nn` runs in a group, which is why we have to use global temporary variables for collecting.

```
  \tl_set_eq:NN #2 \g__mark_first_marks_tl
  \tl_set_eq:NN #3 \g__mark_last_marks_tl
}
```

(*End of definition for* `\mark_get_marks_for_reinsertion:nNN`. *This function is documented on page* 11.)

`\__mark_get_from_splitmarks:` This function is called after we have done a `\vsplit` to update the split marks. It loops through all mark classes to find out if there are marks for this class and if so updates the global tls used for collecting.

```
\cs_new_protected:Npn  \__mark_get_from_splitmarks:  {
  \seq_map_inline:Nn \g__mark_classes_seq
            {
```

[9] It is probably enough to collect everything in a single token list as long as we put the first marks first and the last marks last). But for extra flexibility, I currently use 2 token lists. This might change when it is really clear that this is never needed.

First we to get the last mark for the current class from the material supplied.

```
                \tl_gset:No \g__mark_tmp_tl
                  { \tex_splitbotmarks:D \use:c { c__mark_class_##1_mark } }
```

If this mark doesn't exist then obviously first mark doesn't either, so we do nothing (other than issuing some debugging info).

We have to be a little careful here: something like `\mark_insert:nn{foo}{}` adds an "empty" mark that we should not confuse with the case where there is no mark at all.

When there is no mark at all we get a truly empty `\g__mark_tmp_tl` as a result. This is why we have to make sure that an "empty" mark generated with `\mark_insert:nn` only appears to be empty when it is typeset, but fails the next test (see below how this is done).

```
                \tl_if_empty:NTF \g__mark_tmp_tl
                  {
⟨*trace⟩
                    \__mark_debug:n { \iow_term:e { Marks:~no~ marks~
                        for~ class~ '##1'~\msg_line_context: } }
⟨/trace⟩
                  }
```

If it wasn't empty, i.e., if it had a real value then we use this value for our new last mark instead. This means we put an appropriate `\mark_insert:nn` statement into `\g__mark_last_marks_tl`.

```
                  {
⟨*trace⟩
                    \__mark_debug:n { \iow_term:e { Marks:~ extract~ last~
```

The mark content in `\g__mark_tmp_tl` may contain arbitrary code that may react badly if it is expanded in a write. So we better avoid that expansion, otherwise debugging might generate spurious errors when turned on.

```
                         mark~ for~ class~ '##1'~ =~ \exp_not:o \g__mark_tmp_tl } }
⟨/trace⟩
                    \tl_gput_right:Ne \g__mark_last_marks_tl
                      { \mark_insert:nn {##1} { \__mark_drop_id:o { \g__mark_tmp_tl } } }
```

Because we had a last mark we also have a first mark (which might be the same, but might not be), so we pick that up and add it to the `\g__mark_first_marks_tl` token list. This explains why we first checked for the last mark because that makes the processing faster in case there is none.

```
⟨*trace⟩
                    \__mark_debug:n { \iow_term:e {
                        Marks:~ extract~ first~ mark~ for~ class~ '##1'~ =~
```

Again no expansion for the mark content.

```
                        \exp_not:o {
                          \tex_splitfirstmarks:D
                          \use:c { c__mark_class_##1_mark }
                        }
                    } }
⟨/trace⟩
                    \tl_gput_right:Ne \g__mark_first_marks_tl
                      { \mark_insert:nn {##1}
                        {
```

We better drop the id from the returned value otherwise they will accumulate in the marks when reinserted.

```
                            \__mark_drop_id:o {
                              \tex_splitfirstmarks:D
                              \use:c { c__mark_class_##1_mark }
                            }
                        }
                    }
                }
            }
}
```

(*End of definition for* `\__mark_get_from_splitmarks:`.)

`\g__mark_first_marks_tl`
`\g__mark_last_marks_tl`

These are two global temporary variables used in the code above.

```
\tl_new:N  \g__mark_first_marks_tl
\tl_new:N  \g__mark_last_marks_tl
```

(*End of definition for* `\g__mark_first_marks_tl` *and* `\g__mark_last_marks_tl`.)

`\mark_copy_structure:nn`

This function copies the structure for one region to another, e.g., from `page` to `previous-page` above, or later from `column` to `first-column`, etc.

```
\cs_new_protected:Npn \mark_copy_structure:nn #1#2 {
```

This requires a simple loop through all mark classes copying the token list from one name to the next.

```
  \seq_map_inline:Nn \g__mark_classes_seq
    {
      \tl_gset_eq:cc { g__mark_ #1 _top_   ##1 _tl }
                     { g__mark_ #2 _top_   ##1 _tl }
      \tl_gset_eq:cc { g__mark_ #1 _first_ ##1 _tl }
                     { g__mark_ #2 _first_ ##1 _tl }
      \tl_gset_eq:cc { g__mark_ #1 _last_  ##1 _tl }
                     { g__mark_ #2 _last_  ##1 _tl }
    }
}
```

(*End of definition for* `\mark_copy_structure:nn`. *This function is documented on page* 10.)

`\mark_clear_structure:n`

This function sets the structure of one region back to an initial state, so that all classes return an empty value if queried.

```
\cs_new_protected:Npn \mark_clear_structure:n #1 {
```

This requires a simple loop through all mark classes.

```
  \seq_map_inline:Nn \g__mark_classes_seq
    {
      \tl_gset_eq:cN { g__mark_ #1 _top_   ##1 _tl }
                     \c__mark_empty_tl
      \tl_gset_eq:cN { g__mark_ #1 _first_ ##1 _tl }
                     \c__mark_empty_tl
      \tl_gset_eq:cN { g__mark_ #1 _last_  ##1 _tl }
                     \c__mark_empty_tl
    }
}
```

(*End of definition for* `\mark_clear_structure:n`. *This function is documented on page 11.*)

`\mark_set_structure_to_err:n`
`\__mark_error:n`

A slight variation is to install a fixed error message as the value.

```
\cs_new_protected:Npn \mark_set_structure_to_err:n #1 {
  \seq_map_inline:Nn \g__mark_classes_seq
      {
        \tl_gset:ce { g__mark_ #1 _top_   ##1 _tl } { \__mark_value:nn{?}{\__mark_error:nn {#
        \tl_gset:ce { g__mark_ #1 _first_ ##1 _tl } { \__mark_value:nn{?}{\__mark_error:nn {#
        \tl_gset:ce { g__mark_ #1 _last_  ##1 _tl } { \__mark_value:nn{?}{\__mark_error:nn {#
      }
}
```

Given that this is used in only one place, we could hardwire the argument which would be a bit more compact, but who knows, perhaps we end up with another reason to use this error command elsewhere, so for now we keep the argument.

```
\cs_new_protected:Npn \__mark_error:nn #1#2 {
  \msg_error:nnnn { mark } { invalid-use } {#1} {#2}
}
```

(*End of definition for* `\mark_set_structure_to_err:n` *and* `\__mark_error:n`. *This function is documented on page 10.*)

8.3 Placing and retrieving marks

`\mark_insert:nn`

This function puts a mark for some ⟨*class*⟩ at the current point.

```
\cs_new_protected:Npn \mark_insert:nn #1#2
{
  \seq_if_in:NnTF \g__mark_classes_seq {#1}
      {
```

We need to pass the evaluated argument into the mark but protected commands should not expand including those protected using the `\protect` approach of LaTeX 2_ε. We also disable `\label` and the like.[10]

At this point the code eventually should get a public (and a kernel) hook instead of a set of hardwired settings.

```
        \group_begin:
```

Within the group we alter some comments, e.g, `\label` or `\index`, to do the right at this point. This is done in the kernel hook `\@kernel@before@insertmark` which is followed by the public hook `insertmark` that can be used by packages to augment or alter that setup as necessary.

```
          \@kernel@before@insertmark
          \hook_use:n { insertmark }
          \unrestored@protected@xdef \g__mark_tmp_tl
              {
```

To ensure that marks are unique we insert a hidden sequence marker at the beginning of the content of the mark containing the sequence number of the mark.

```
                \__mark_value:nn{ \int_use:N\g__mark_int }{#2}
              }
⟨*trace⟩
          \__mark_debug:n{ \iow_term:e { Marks:~ set~#1~<-~
```

[10] Straight copy from `latex.ltx` but is this even correct? At least a label in a running header makes little sense if it get set several times! Maybe that needs looking at in the 2e kernel.

```
                ’\tl_to_str:V \g__mark_tmp_tl’ ~ \msg_line_context: } }
⟨/trace⟩
            \tex_marks:D \use:c { c__mark_class_ #1 _mark }
              {
```

Here is the trick to avoid truly empty marks: if the result from the above processing is empty we add something which eventually becomes empty, but not immediately; otherwise we just put `\g__mark_tmp_tl` in.

```
% This is no longer needed with 1.0f
%               \tl_if_empty:NTF \g__mark_tmp_tl
%                 { \exp_not:n { \prg_do_nothing: } }
%                 { \exp_not:o { \g__mark_tmp_tl } }
                \exp_not:o { \g__mark_tmp_tl }
              }
          \group_end:
```

A mark introduces a possible break point and in certain situations that should not happen in vertical mode in LaTeX. This may need some checking and possibly cleanup ....

```
          \if@nobreak\ifvmode\nobreak\fi\fi
        }
```

If the mark class was not known, raise an error.

```
        {
          \msg_error:nne { mark } { unknown-class }
            { \tl_to_str:n {#1} }
        }
}
```

(*End of definition for* `\mark_insert:nn`. *This function is documented on page* 3.)

`\__mark_value:nn`

A hidden marker is placed into every mark added by `\mark_insert:nn`. It will not show up in the output but its argument (a counter value that is incremented) makes all marks unique so the test for “equal” is not fooled by two different marks having the same mark text.

```
\cs_new_protected:Npn \__mark_value:nn #1#2 { #2 }
```

(*End of definition for* `\__mark_value:nn`.)

`\@kernel@before@insertmark`
`insertmark`

By default `\label`, `\index`, and `\glossary` do nothing when the mark is inserted.

```
\int_new:N \g__mark_int
\cs_new:Npn \@kernel@before@insertmark {
            \cs_set_eq:NN \label    \scan_stop:
            \cs_set_eq:NN \index    \scan_stop:
            \cs_set_eq:NN \glossary \scan_stop:
```

We count each mark and use that to place a hidden marker in front of the mark text. To ensure that there is no overflow (very unlikely but you never know) we restart every 100000 marks. Thus, if somebody puts more than that number of marks on a single page you could construct a scenario in which that approach fails.

```
            \int_compare:nNnTF \g__mark_int < {99999}
                { \int_gincr:N \g__mark_int }
                { \int_gzero:N \g__mark_int }

}
```

The public hook to augment the setup.

```
\hook_new:n {insertmark}
```

(*End of definition for* `\@kernel@before@insertmark` *and* `insertmark`.)

`\mark_use_top:nn`
`\mark_use_first:nn`
`\mark_use_last:nn`

To retrieve the first, last or top region mark, we grab the appropriate value stored in the corresponding token list variable and pass its contents back. These functions should be used only in output routines and only after `\mark_update_structure_from_material:nn` has acted, otherwise their value will be wrong.

```
\cs_new:Npn \mark_use_first:nn #1#2 { \__mark_use_check:nnn { g__mark_#1_first_#2_tl } {#1} {
\cs_new:Npn \mark_use_last:nn #1#2 { \__mark_use_check:nnn { g__mark_#1_last_#2_tl } {#1} {#2
\cs_new:Npn \mark_use_top:nn #1#2 { \__mark_use_check:nnn { g__mark_#1_top_#2_tl } {#1} {#2}
```

If used with an unknown class or region these commands will generate an error. If that happens in an expandable context then the error generation is delayed (e.g., if used in a `\section`) and happens when the code is finally used in typesetting, e.g., in the TOC or a running header. If used in a `\typeout` you only see something like `\__mark_error:n{page}`. This is not too good, but probably better than low-level errors, I guess, and I don't want to use an expandable error because of the size restrictions in such error messages.

```
\cs_new:Npn \__mark_use_check:nnn #1#2#3 {
  \tl_if_eq:cNTF {#1} \relax
     { \__mark_error:nn {#2} {#3} }
     { \__mark_drop_id:v {#1} }
}
```

Each mark starts with an id and while the id does not print it is nevertheless better to remove it when returning the mark, so that downstream manipulation of the data doesn't have to deal with it. This is what the `\exp_not:o` accomplishes.

```
\cs_new:Npn \__mark_drop_id:n #1 { \exp_not:o { #1 } }
\cs_generate_variant:Nn \__mark_drop_id:n { o, v }
```

(*End of definition for* `\mark_use_top:nn`, `\mark_use_first:nn`, *and* `\mark_use_last:nn`. *These functions are documented on page* 4.)

8.4 Comparing mark values

`\mark_if_eq:nnnnTF`
`\mark_if_eq:nnnnnnTF`

Test if in a given region (`#1`) for a given class (`#2`) the marks in position `#3` and `#4` (top, first, or last) are identical

```
\prg_new_conditional:Npnn \mark_if_eq:nnnn #1#2#3#4 { T , F , TF }
{
  \tl_if_eq:ccTF { g__mark_ #1 _#3_ #2 _tl }
                 { g__mark_ #1 _#4_ #2 _tl }
                 \prg_return_true:
                 \prg_return_false:
}
```

The fully general test (with two triplets of the form ⟨*region*⟩, ⟨*class*⟩, and ⟨*position*⟩) is this:

```
\prg_new_conditional:Npnn \mark_if_eq:nnnnnn #1#2#3#4#5#6 { T , F , TF }
{
  \tl_if_eq:ccTF { g__mark_ #1 _#3_ #2 _tl }
                 { g__mark_ #4 _#6_ #5 _tl }
                 \prg_return_true:
```

```
311                     \prg_return_false:
312 }
```

(*End of definition for* `\mark_if_eq:nnnnTF` *and* `\mark_if_eq:nnnnnnTF`. *These functions are documented on page 4.*)

8.5 Messages

Mark errors are LaTeX kernel errors:

```
313 \prop_gput:Nnn \g_msg_module_type_prop { mark } { LaTeX }

314 \msg_new:nnnn { mark } { class-already-defined }
315   { Mark~class~'#1'~already~defined }
316   {
317     \c__msg_coding_error_text_tl
318     LaTeX~was~asked~to~define~a~new~mark~class~called~'#1':~
319     this~mark~class~already~exists.
320     \c__msg_return_text_tl
321   }

322 \msg_new:nnnn { mark } { unknown-class }
323   { Unknown~mark~class~'#1'. }
324   {
325     \c__msg_coding_error_text_tl
326     LaTeX~was~asked~to~manipulate~a~mark~of~class~'#1',~
327     but~this~class~of~marks~does~not~exist.
328   }
```

The next error can also happen if the mark class is unknown, so this should perhaps be separated into two different errors.

```
329 \msg_new:nnnn { mark } { invalid-use }
330   { Mark~region~'#1'~not~usable~or~class~'#2'~unknown }
331   {
332     \c__msg_coding_error_text_tl
333     The~region~'#1'~is~either~not~known~or~data~for~it~
334     still~needs~to~be~assembled,~e.g.,~last-column~
335     while~building~the~first-column.~
336     Also~possible:~the~class~name~'#2'~is~misspelled.
337     \c__msg_return_text_tl
338   }
```

8.6 Debugging the mark structures

Code and commands in this section are not final, it needs more experimentation to see what kind of tracing information is going to be useful in practice. For now the tracing is mainly meant to be used for code testing and not so much for application testing.

It is quite likely that the commands and the behavior of the tracing might change in the future once we gained some experience with it.

`\g__mark_debug_bool` Holds the current debugging state.

```
339 \bool_new:N \g__mark_debug_bool
```

(*End of definition for* `\g__mark_debug_bool`.)

\mark_debug_on:
\mark_debug_off:
__mark_debug:n
__mark_debug_gset:

Turns debugging on and off by redefining `\__mark_debug:n`.

```
\cs_new_eq:NN \__mark_debug:n \use_none:n
\cs_new_protected:Npn \mark_debug_on:
  {
    \bool_gset_true:N \g__mark_debug_bool
    \__mark_debug_gset:
  }
\cs_new_protected:Npn \mark_debug_off:
  {
    \bool_gset_false:N \g__mark_debug_bool
    \__mark_debug_gset:
  }
\cs_new_protected:Npn \__mark_debug_gset:
  {
    \cs_gset_protected:Npe \__mark_debug:n ##1
      { \bool_if:NT \g__mark_debug_bool {##1} }
  }
```

(*End of definition for* `\mark_debug_on:` *and others. These functions are documented on page* 7.)

\DebugMarksOn
\DebugMarksOff

CamelCase commands for debugging.

```
\cs_new_eq:NN \DebugMarksOn   \mark_debug_on:
\cs_new_eq:NN \DebugMarksOff  \mark_debug_off:
```

(*End of definition for* `\DebugMarksOn` *and* `\DebugMarksOff`. *These functions are documented on page* 7.)

__mark_class_status:nnn

Shows the mark values across all regions for one mark class (`#2`).

The first argument gives some ⟨*info*⟩ to help in identifying where the command was called, the second is the class and the third holds the number of `mcol-...` we should display: inside a `multicols` environment this will be `\col@number`, in LaTeX's normal output routines it will be `0`.

```
⟨*trace⟩
\cs_new_protected:Npn \__mark_class_status:nnn #1#2#3 {
   \typeout{ Marks:~#2~ #1:}
   \__mark_region_status:nnn  {#2}{ page~ (previous) }  { previous-page }
   \__mark_region_status:nnn  {#2}{ page~ (current)~ }  { page }
   \__mark_region_status:nnn  {#2}{ column~ (previous) }{ previous-column }
   \__mark_region_status:nnn  {#2}{ column~ (current)~ }{ column }
   \__mark_region_status:nnn  {#2}{ column~ (first) }   { first-column }
   \__mark_region_status:nnn  {#2}{ column~ (last)~ }   { last-column }
```

Then finish by displaying a subset of the `mcol-...` regions: none (`0`) in the standard LaTeX output routine and `\col@number` within a `multicols` environment.

```
   \int_step_inline:nn {#3}
     {
       \__mark_region_status:nnn  {#2}{ column~ (##1)~ }   { mcol-##1 }
     }
}
```

(*End of definition for* `\__mark_class_status:nnn`.)

__mark_region_status:nnn

Display the top, first, and last mark of a region unless none of them exist or all of them are empty.

```
\cs_new_protected:Npn \__mark_region_status:nnn #1#2#3 {
```

```
  \group_begin:
  \cs_set:Npn \__mark_value:nn ##1##2{ \exp_not:n{ {##1} ~ ##2 } }
  \tl_if_exist:cT  { g__mark_#3_last_ #1 _tl }
     {
       \tl_if_eq:cNF { g__mark_#3_last_ #1 _tl } \c__mark_empty_tl
          {
            \typeout{\@spaces #2 =
               ~|~ \use:c { g__mark_#3_top_ #1 _tl }    ~|~
                   \use:c { g__mark_#3_first_ #1 _tl }  ~|~
                   \use:c { g__mark_#3_last_ #1 _tl }   ~|
            }
          }
     }
  \group_end:
}
```

(*End of definition for* `\__mark_region_status:nnn`.)

`\__mark_status:nn` Show a snapshot of all mark class values across all regions. The first argument is a string to identify the output, the second argument is the number of `mcol-...` regions to show. Outside of a `multicols` environment this is normally set to 0.

```
\cs_new_protected:Npn \__mark_status:nn #1#2
  {
    \seq_map_inline:Nn \g__mark_classes_seq
      { \__mark_class_status:nnn {#1} {##1} {#2} }
  }
⟨/trace⟩
```

(*End of definition for* `\__mark_status:nn`.)

`\ShowMarksAt` Debugging helper that displays a snapshot of all known mark structures. The first argument is a text string that is displayed to help identifying when the snapshot was made. The optional second one determines how many `mcol-...` regions are displayed (by default 4).

This may not stay like this (or at all), which is why it isn't yet documented as an official command.

```
\NewDocumentCommand \ShowMarksAt {m O{4} } {
⟨*trace⟩
     \__mark_debug:n { \__mark_status:nn {#1}{#2} }
⟨/trace⟩
}
```

(*End of definition for* `\ShowMarksAt`. *This function is documented on page* ??.)

8.7 Designer-level interfaces

`\NewMarkClass` `\InsertMark` These two are identical to the L3 programming layer commands.

```
\cs_new_eq:NN  \NewMarkClass \mark_new_class:n
\@onlypreamble \NewMarkClass

\cs_new_eq:NN \InsertMark  \mark_insert:nn
```

(*End of definition for* `\NewMarkClass` *and* `\InsertMark`. *These functions are documented on page* 3.)

\TopMark
\FirstMark
\LastMark

The following commands take an optional argument that defaults to page. There is no checking that the region is actually valid. If not there is simply an empty return.

```
\NewExpandableDocumentCommand \FirstMark { O{page} m }
                     { \mark_use_first:nn {#1}{#2} }
\NewExpandableDocumentCommand \LastMark { O{page} m }
                     { \mark_use_last:nn {#1}{#2} }
\NewExpandableDocumentCommand \TopMark { O{page} m }
                     { \mark_use_top:nn {#1}{#2} }
```

(*End of definition for* `\TopMark`, `\FirstMark`, *and* `\LastMark`. *These functions are documented on page* 4.)

\IfMarksEqualTF
\IfMarksEqualT
\IfMarksEqualF

We only provide CamelCase commands for the case with one region (optional) and one class. One could think of also providing a version for the general case with several optional arguments, but use cases for this are most likely rare, so not done yet.

```
\NewExpandableDocumentCommand \IfMarksEqualTF {O{page}mmm} {
  \mark_if_eq:nnnnTF {#1}{#2}{#3}{#4}
}
\NewExpandableDocumentCommand \IfMarksEqualT {O{page}mmm} {
  \mark_if_eq:nnnnT {#1}{#2}{#3}{#4}
}
\NewExpandableDocumentCommand \IfMarksEqualF {O{page}mmm} {
  \mark_if_eq:nnnnF {#1}{#2}{#3}{#4}
}
```

(*End of definition for* `\IfMarksEqualTF`, `\IfMarksEqualT`, *and* `\IfMarksEqualF`. *These functions are documented on page* 4.)

9 LaTeX 2ε integration

9.1 Core LaTeX 2ε integration

__mark_update_singlecol_structures:

This command updates the mark structures if we are producing a single column document.

```
\cs_new_protected:Npn \__mark_update_singlecol_structures: {
```

First we update the `page` region (which also updates the `previous-page`.

The `\@outputbox` is normally in `\vbox` in LaTeX but we can't take that for granted (an amsmath test document changed it to an `\hbox` just to trip me up) so we are a little careful with unpack now.

```
  \box_if_vertical:NTF \@outputbox
      {
        \mark_update_structure_from_material:nn {page}
           { \vbox_unpack:N  \@outputbox }
      }
      {
        \mark_update_structure_from_material:nn {page}
           { \hbox_unpack:N  \@outputbox }
      }
```

Then we provide the necessary updates for the aliases.

```
  \mark_copy_structure:nn {previous-column}{previous-page}
  \mark_copy_structure:nn {column}{page}
  \mark_copy_structure:nn {first-column}{page}
  \mark_copy_structure:nn {last-column}{page}
⟨*trace⟩
% move this into status itself?
     \__mark_debug:n
         {
           \__mark_status:nn
                { in~ OR~ (
                  \legacy_if:nTF {@twoside}
                         { twoside-
                           \int_if_odd:nTF \c@page
                               { odd }{ even }
                         }
                         { oneside }
                         )
                }
                {0}
         }
⟨/trace⟩
}
```

(*End of definition for* `\__mark_update_singlecol_structures:`.)

`\__mark_update_dblcol_structures:` This commands handles the updates if we are doing two-column pages.

```
\cs_new_protected:Npn \__mark_update_dblcol_structures: {
```

First we update the `column` and `previous-column` regions using the material assembled in `\@outputbox`.

```
  \box_if_vertical:NTF \@outputbox
      {
        \mark_update_structure_from_material:nn {column}
           { \vbox_unpack:N  \@outputbox }
      }
      {
        \mark_update_structure_from_material:nn {column}
           { \hbox_unpack:N  \@outputbox }
      }
```

How we have to update the alias regions depends on whether or not `\@opcol` was called to process the first column or to produce the completed page

```
  \legacy_if:nTF {@firstcolumn}
    {
```

If we are processing the first column then `column` is our `first-column` and there is no `last-column` yet, so we make those an error.

```
      \mark_copy_structure:nn {first-column}{column}
      \mark_set_structure_to_err:n {last-column}
    }
    {
```

If we produce the completed page then the `first-column` is the same as the new `previous-column`. However, the structure should already be correct if you think about it (because is was set to `column` last time which is now the `previous-column`), thus there is no need to make an update.

```
%     \mark_copy_structure:nn {first-column}{previous-column}
```

However, we now have a proper `last-column` so we assign that.

```
      \mark_copy_structure:nn {last-column}{column}
```

What now remains doing is to update the `page` and `previous-page` regions. For this we have to copy the settings in `page` into `previous-page` and then update `page` such that the top and first marks are taken from the `first-column` region and the last marks are taken from the `last-column` region. All this has to be done for all mark classes so we loop over our sequence.

Note that one loop is needed if we arrange the copy statements in a suitable way.

```
      \seq_map_inline:Nn \g__mark_classes_seq
        {
```

The `previous-page` updates need to come before the updates for `page` region because otherwise the values to copy are already overwritten. necessary values.

```
          \tl_gset_eq:cc { g__mark_previous-page_top_   ##1 _tl }
                         { g__mark_page_top_           ##1 _tl }
          \tl_gset_eq:cc { g__mark_previous-page_first_ ##1 _tl }
                         { g__mark_page_first_         ##1 _tl }
          \tl_gset_eq:cc { g__mark_previous-page_last_  ##1 _tl }
                         { g__mark_page_last_          ##1 _tl }
```

To update the `top` we only have to copy what is in `first-column`:

```
          \tl_gset_eq:cc { g__mark_page_top_          ##1 _tl }
                         { g__mark_first-column_top_  ##1 _tl }

```

Updating the `first` mark for the `page` region is more complicated. We first have to find out of there is any mark in the first column (this can be done by comparing the `top` and the `first` mark of that region).

```
          \tl_if_eq:ccTF { g__mark_first-column_top_   ##1 _tl }
                         { g__mark_first-column_first_ ##1 _tl }
            {
```

If there is no mark in the first column we copy the first mark of the last column. If that doesn't contain a mark we still get the right result because the first mark is then equal to the top mark.

```
              \tl_gset_eq:cc { g__mark_page_first_        ##1 _tl }
                             { g__mark_last-column_first_ ##1 _tl }
            }
            {
```

On the other hand, if there is a mark in the first column we copy over the `first` mark from that column.

```
              \tl_gset_eq:cc { g__mark_page_first_         ##1 _tl }
                             { g__mark_first-column_first_ ##1 _tl }
            }
```

The logic for the `last` page mark is again simple, we can just copy the value in the `last` mark of the last column. If that column doesn't contain any marks, then the value in `last` will be automatically the same as the `last` from the first column.

```
          \tl_gset_eq:cc { g__mark_page_last_        ##1 _tl }
                         { g__mark_last-column_last_ ##1 _tl }
        }
    }
⟨*trace⟩
    \__mark_debug:n
        {
          \__mark_status:nn
             { in~ OR~ (
               \legacy_if:nTF {@twoside}
                      { twoside-
                        \int_if_odd:nTF \c@page
                            { odd }{ even }
                      }
                      { oneside }
                      \space
                      \legacy_if:nTF {@firstcolumn}
                             { first~ }{ second~ }
                             column )
             }
             {0}
         }
⟨/trace⟩
}
```

(*End of definition for* `\__mark_update_dblcol_structures:`.)

9.2 Other LaTeX 2ε output routines

This section will cover support for packages that alter the LaTeX output routine (as necessary). The support for multicol (for now) is handled directly in that package.

```
⟨@@=⟩
```

`\@expl@@@mark@update@singlecol@structures@@`

```
\cs_new_eq:NN  \@expl@@@mark@update@singlecol@structures@@
               \__mark_update_singlecol_structures:
```

(*End of definition for* `\@expl@@@mark@update@singlecol@structures@@`.)

`\@expl@@@mark@update@dblcol@structures@@`

```
\cs_new_eq:NN  \@expl@@@mark@update@dblcol@structures@@
               \__mark_update_dblcol_structures:
```

(*End of definition for* `\@expl@@@mark@update@dblcol@structures@@`.)

9.3 Rollback information

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{ltmarks}%
⟨latexrelease⟩                  {Undo~Marks~handling}
⟨latexrelease⟩
```

We keep the interface commands around even if we roll back in case they are used in packages that don't roll back. Not likely to do a lot of good, but then there is not much we can do, but this at least they won't give unknown csname errors.

```
⟨latexrelease⟩\DeclareRobustCommand \NewMarkClass[1]{}
⟨latexrelease⟩\DeclareRobustCommand \InsertMark[2]{}
⟨latexrelease⟩\RenewExpandableDocumentCommand \FirstMark { O{} m } { }
⟨latexrelease⟩\RenewExpandableDocumentCommand \LastMark { O{} m } { }
⟨latexrelease⟩\RenewExpandableDocumentCommand \TopMark { O{} m } { }
⟨latexrelease⟩\RenewExpandableDocumentCommand \IfMarksEqualTF { O{} mmm }{ }
⟨latexrelease⟩
```

Same here, this avoided extra roll back code in the OR.

```
⟨latexrelease⟩\let \@expl@@@mark@update@singlecol@structures@@ \relax
⟨latexrelease⟩\let \@expl@@@mark@update@dblcol@structures@@ \relax
⟨latexrelease⟩
⟨latexrelease⟩
⟨latexrelease⟩\EndModuleRelease
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
```

Reset module prefix:

```
⟨@@=⟩
```